

ti&m

Agentic Coding for Software Engineers

Thomas Treuthardt

Eric Obermühlner

Zürich, Mai 2026

ti&m

Agenda

Time	Topic
09:00 – 09:15	Introduction
09:15 – 09:30	Agentic Coding
09:30 – 10:00	AI Fundamentals
10:00 – 10:15	<i>Break</i>
10:15 – 10:30	Copilot CLI vs. Claude Code: Overview
10:30 – 11:30	Copilot CLI: Instructions & Context
11:30 – 12:30	Copilot CLI: Skills, Agents, MCP
12:30 – 13:30	<i>Lunch</i>
13:30 – 14:00	Spec Driven Development
14:00 – 14:30	Exercise: Project Exploration
14:30 – 14:45	<i>Break</i>
14:45 – 15:45	Exercise: Spec Driven Development
15:45 – 16:00	Reflection Exercise
16:00 – 16:30	Q&A
16:30 – 17:00	Agentic Coding @ Raiffeisen

Introduction

Agentic Coding for Software Engineers

Who are we ?

- Thomas Treuthardt — Senior Software Architect, ti&m
- Eric Obermühlner — Senior Software Architect, ti&m

Current AI Experience

- Who uses AI coding agents daily?
- Who uses Spec Driven Development?
- Who thinks they are faster using AI?

This check-in helps us adjust the depth and pace of each topic based on your experience.

Course Goals

- Understand AI-driven software development
- Get started with GitHub Copilot CLI and Claude Code
- Agentic Coding: Context Management
- Agentic Coding: Skills
- Agentic Coding: Agents
- Agentic Coding: MCP
- Spec Driven Development workflow

The Five Levels of AI-Assisted Development

Level	Name	AI Role
1	AI Code Assistance	On-demand chat (ChatGPT, Claude) – dev integrates manually
2	AI Code Generation	IDE plugins suggest code inline (Copilot, JetBrains AI)
3	Agentic AI Development	Agent reads, writes, runs tests autonomously (Claude Code, Cursor, Copilot CLI)
4	Spec-Driven Development	Human + AI co-author a spec; agent implements (same tools, higher discipline)
5	Autonomous Product Evolution	AI builds entire products

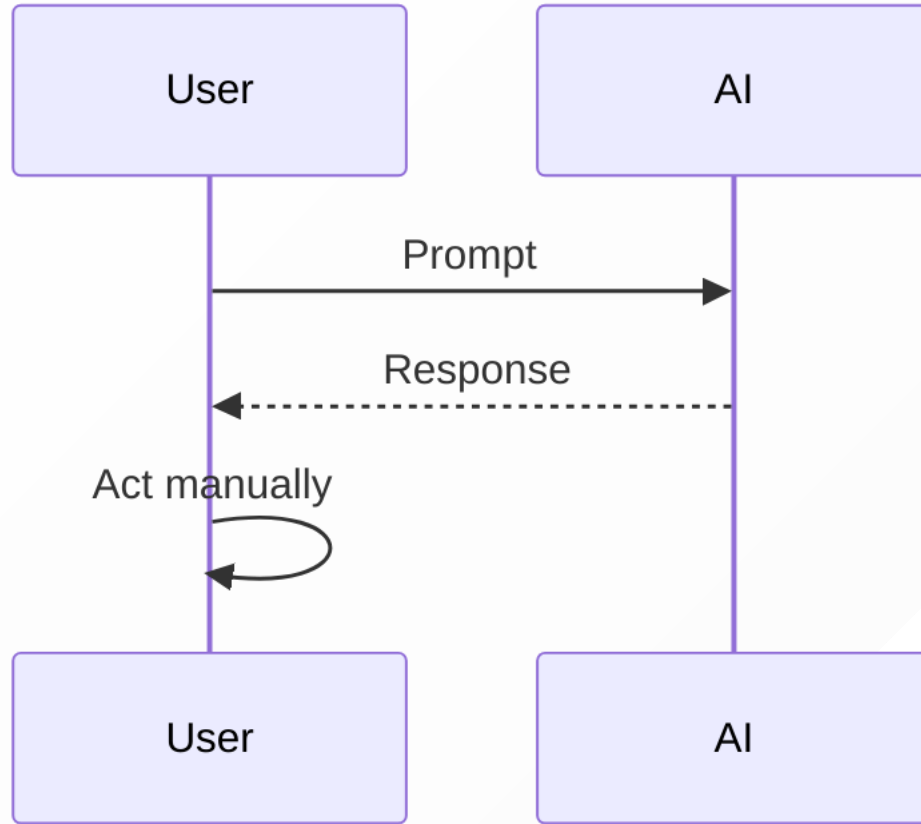
AI Readiness: Prerequisites for Success

- **Clean architecture** — chaotic codebases produce chaotic AI output
- **Automated quality gates** — linters, type checkers, and tests catch AI mistakes before they reach production
- **Fast Build & Test** — fast feedback loop improves the AI workflow
- **Context** — README, architecture docs, and instruction files give AI the context it needs
- **Security & isolation** — containerize or sandbox agents to mitigate risk
- **Team skills** — code review shifts from "did you write this right?" to "did you verify AI output?"

Level 4 is currently the most realistic target for regulated environments

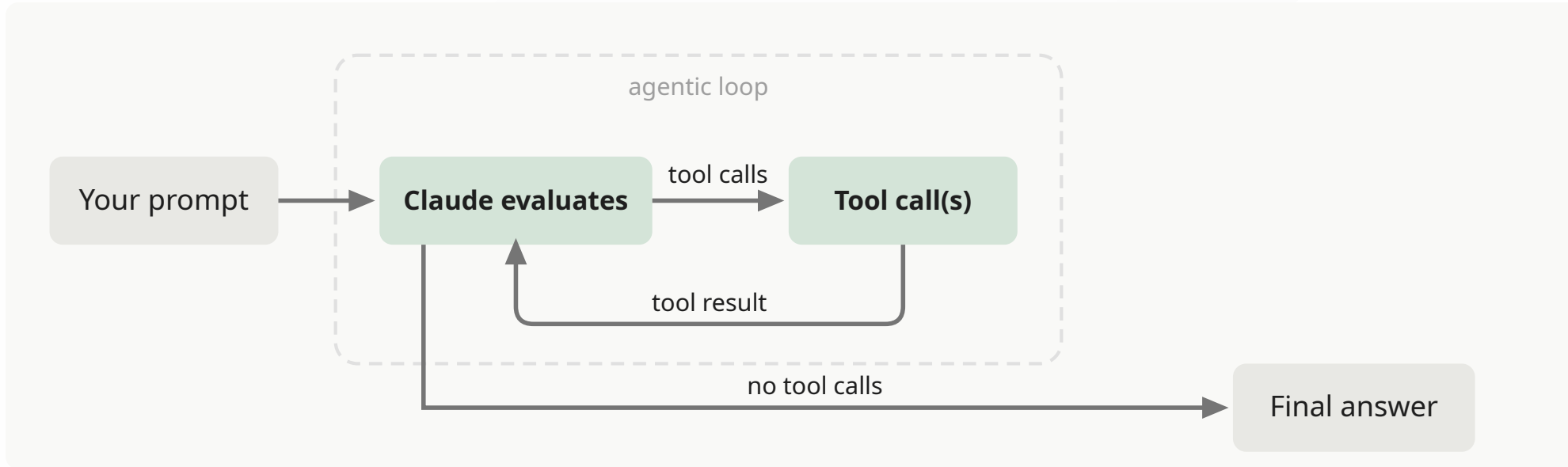
Chat-Based Interaction

User prompt → LLM response → User reads and acts manually



Agentic Coding Loop

Tool use and feedback loop are the core of agentic coding.



Important observations:

- Tool results are the feedback loop that allows the agent to iteratively correct mistakes.
- The agentic loop can iterate many times. Without accurate prompt the agent will make assumptions and produce incorrect results.

The loop runs autonomously – the human sets the goal and reviews the result.

Recommended Mindset

Treat the AI like a smart junior dev: review its code, don't trust its confidence.

- **Junior Developer Analogy** — Treat AI as knowledgeable but inexperienced
- **Embrace the Tool** — View AI like your IDE — powerful when used correctly, but requiring skill to master
- **Context is Your Main Lever** — Vague input → vague output based on assumptions.
The more specific your instructions and context, the better the result.
- **Professional Standards** — Apply the same code review, testing, and validation standards
- **Expect Imperfection** — Plan for verification, testing, and iteration as AI will make mistakes
- **You Own the Result** — You commit, ship, and are accountable for every line the AI wrote

Your job shifts from writing code to providing context, reviewing output, and owning the result.

Agentic AI in the SDLC – Risks

Implementing agentic AI in software development introduces significant risks that teams must understand.

- **Unrealistic Expectations** – Hype and marketing create false expectations about agentic coding capabilities, leading to inappropriate use cases and project failures
- **Code Quality Degradation** – Incorrect usage can produce unmaintainable, insecure, or functionally broken applications that create long-term technical debt
- **Skill Development Gap** – Junior developers may skip fundamental learning, lacking essential experience in professional software development practices
- **Team Resistance** – Developer skepticism can create adoption barriers, with negative experiences spreading quickly through engineering teams
- **Cost Escalation** – Enterprise token-based pricing can result in unexpected bills of thousands of CHF within short periods without proper usage controls

Agentic AI amplifies both good and bad development practices – success requires proper training, realistic expectations, and robust safeguards.

Concerns of Software Engineers

- **"Almost Right" Frustration** — The most common complaint: AI output looks correct but requires manual fixes, creating a hidden productivity tax
- **Debugging Overhead** — Speed gains in code generation are partially offset by increased time spent debugging and validating AI output
- **Mixed Productivity Results** — Perceived speed-ups don't always hold up under measurement; gains depend heavily on task type and developer experience
- **Trust Gap** — Some surveys indicate a significant portion of developers don't trust AI-generated code
- **Maintenance Burden** — Generated code can be harder to maintain, especially if the original developer doesn't fully understand it

How This Course Addresses These Concerns

- **Context discipline** — Project instructions and structured prompts reduce "almost right" output
- **Quality Gates & Reviews** — Guardrails prevent silent mistakes from reaching production
- **SDD** — Spec Driven Development minimizes AI assumptions

AI Fundamentals

Large Language Models

Choose models based on specific strengths — coding, reasoning, multimodal, or openness.

- **What they are** — Deep learning neural networks trained on vast text data; they predict the next token given context
- **Tokens** — The fundamental units LLMs process; one word can be multiple tokens.
Token limits define the **context window** — the maximum text a model can process at once
- **Context Window** — Think of it as working memory: everything the model can "see" in one request — your prompt, history, files, tool results
- **Training Cutoff** — Knowledge is frozen at training time; models don't know about recent events unless provided via context or tools

Understanding tokens and context windows is essential — pricing, quality, and behavior all depend on how you fill the window.

Vibe Coding vs. Agentic Engineering

Vibe Coding works for prototypes; production software requires structured prompting and rigorous validation.

Vibe Coding (*Karpathy, Feb 2025*)

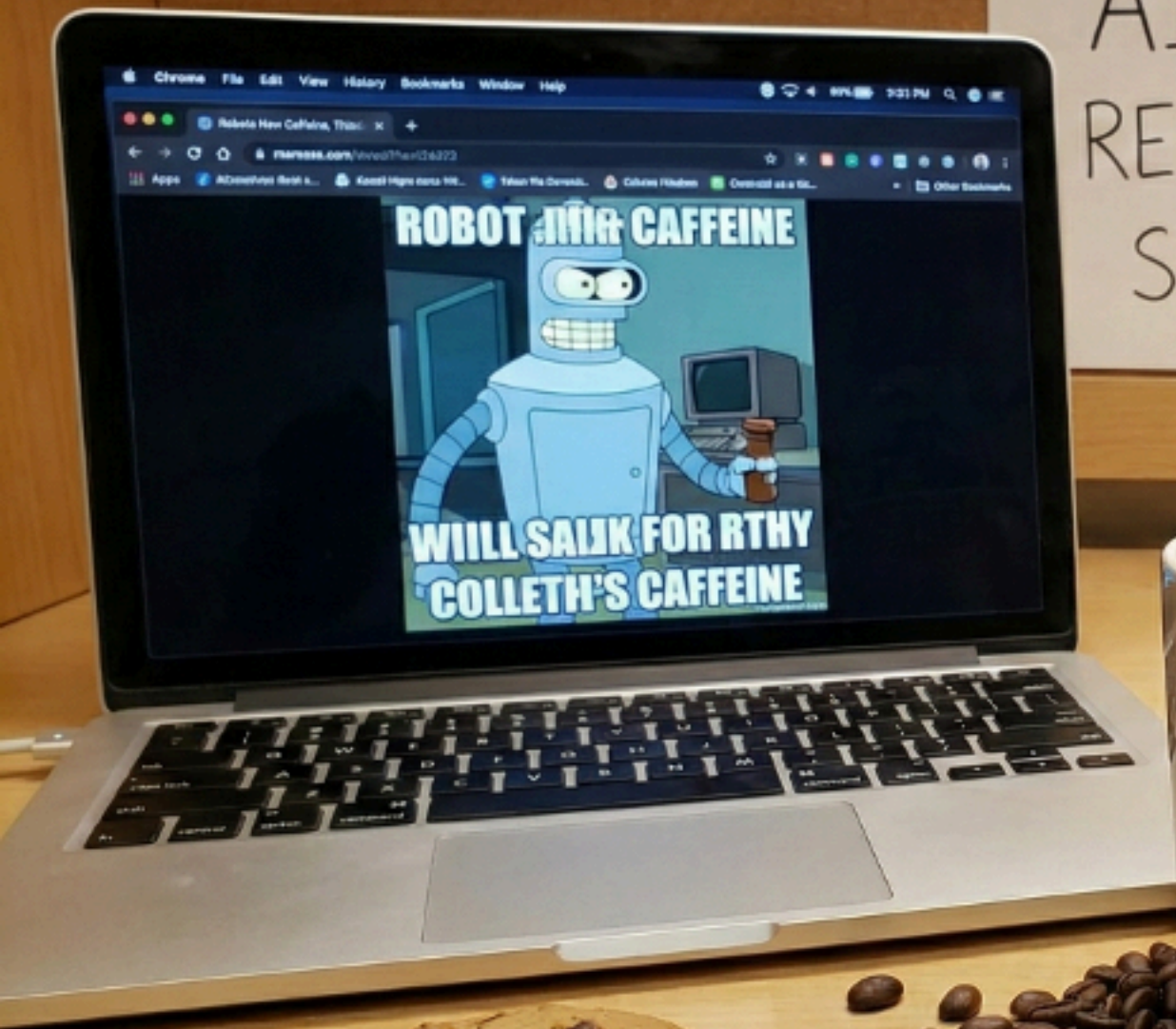
- Intuitive, natural language prompts – no structured framework
- Result is a blackbox
- Lacks validation for maintainable production code
- Excellent for quick experimentation and throwaway code

Agentic Engineering (*Karpathy, Feb 2026*)

AI does the implementation, the human owns the architecture, quality, and correctness. – Addy Osmani

- **Human Owns Architecture** – Engineer defines the plan, scope, and constraints; AI handles implementation
- **Specify Before You Generate** – Write/review specs, define acceptance criteria, set boundaries before the agent codes
- **Review Like a Tech Lead** – Question design choices. Reject what you wouldn't approve in a PR

AI LAB: REFUELING STATION



GitHub Copilot CLI vs. Claude Code

Agentic Coding for Software Engineers

GitHub Copilot – Three Modes

GitHub Copilot is not one tool – it has three distinct modes. Don't conflate them.

Inline Suggestions (*IDE*)

Autocomplete-style code completions as you type.

The original Copilot experience – assistive, not agentic.

Agent Mode (*IDE*)

Autonomous editing within VS Code / JetBrains.

Plan mode + multi-file edits inside the IDE.

Copilot CLI (*Terminal*)

Terminal agent (`copilot`).

Agentic tool use: reads files, runs commands, edits code – like Claude Code.

Copilot CLI vs. Claude Code

Dimension	Copilot CLI	Claude Code
Interface	Terminal (<code>copilot</code>)	Terminal (<code>claude</code>)
Models	Multi-vendor: Claude, GPT — user's choice	Anthropic only: Sonnet, Opus, Haiku
Context window	Depends on model; auto-compaction; <code>/context</code>	Depends on model; Up to 1M tokens; auto-compaction; <code>/context</code>
Project instructions	<code>.github/copilot-instructions.md</code> , path-scoped <code>*.instructions.md</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code>	<code>CLAUDE.md</code> (global, project root, subdirs), path-scoped <code>.claude/rules/*.md</code>
MCP	Full support; built-in GitHub MCP; <code>/mcp add</code>	Full support; no built-in servers; <code>claude mcp add</code>
Skills	<code>SKILL.md</code> format; reads <code>.github/skills/</code> , <code>.claude/skills/</code>	<code>SKILL.md</code> format; <code>.claude/skills/</code> ; <code>claude plugin install</code>
Agents / subagents	Built-in: explore, research, task, code-review; custom agents; <code>/fleet</code> for parallelism	Built-in: Explore, Plan, General-purpose; custom subagents; <code>/batch</code> for parallelism
Permissions	CLI options: <code>--allow-tool</code> , ... ; stored in <code>~/.copilot/permissions-config.json</code>	Granular: <code>deny > ask > allow</code> per tool/pattern in <code>.claude/settings.json</code>
Hooks	Execute scripts on ~6 hooks: <code>sessionStart</code> , <code>sessionEnd</code> , <code>userPromptSubmitted</code> , <code>preToolUse</code> , <code>postToolUse</code> , <code>errorOccurred</code>	Execute scripts or prompts on ~21 hooks: <code>PreToolUse</code> , <code>PostToolUse</code> , <code>SubagentStop</code> , <code>PreCompact</code> , ...

Copilot CLI in Practice

Instructions, Context & Permissions

with Claude Code migration notes

GitHub Copilot CLI – Basics

Copilot CLI

Start with `copilot` in your terminal.

Claude Code

Start with `claude` in your terminal.

- Use natural language to ask questions, get code explanations, or request changes.
- Use `/` commands for built-in tools or custom slash commands.
- Use `@path/to/file` to explicitly include a file in context; tab completion works

GitHub Copilot CLI – Modes

Copilot CLI

Toggle with **Shift+Tab** between three modes:

- **Ask/Execute Mode (default)** – Copilot asks for your approval before each tool that could modify or execute files
- **Plan Mode** – Copilot builds a plan and asks clarifying questions before writing any code; also **/plan** command
- **Autopilot Mode** – Copilot runs fully autonomously until the task is completed;

Claude Code

Toggle with **Shift+Tab** between four modes:

- **Normal (default)** – Asks for confirmation for every file edit
- **Auto-Accept** – Accepts all edits without confirmation
- **Plan Mode** – Creates a plan without editing any files. Then asks for approval before switching to execution mode.
- **Auto Mode** – Runs without prompts, but a safety classifier blocks risky actions first

Copilot Autopilot mode will prevent all requests and questions to the user!

Permissions in Copilot CLI

Allow and deny specific tools to balance autonomy and safety.

Tool permissions via CLI flags or agent profiles:

- `--allow-tools='write'` – Allow to write files without confirmation
- `--deny-tools='shell(git push)'` – Deny the agent from pushing to git
- `--available-tools='shell,read'` – Limit the agent to only shell and read tools
- `--allow-all-tools` – Allow all tools without confirmation (use with caution!)

Stored in `~/.copilot/permissions-config.json`

```
{
  "locations": {
    "/home/eric/dev/my-example-project": {
      "tool_approvals": [
        {
          "kind": "write"
        },
        {
          "kind": "commands",
          "commandIdentifiers": [
            "mvn -f"
          ]
        }
      ]
    }
  ]
}
```

Permissions in Claude Code

Fine-grained `allow / ask / deny` permission model per tool using `*` patterns:

```
{
  "permissions": {
    "allow": ["Bash(npm run lint)"],
    "deny": ["Bash(curl *)", "Read(.env)"]
  }
}
```

Stored in `.claude/settings.json` — committed to git and shared with the team.

Project Instructions

Tell the agent your project conventions — once, not in every prompt.

Copilot CLI

File	Scope
<code>~/.copilot/copilot-instructions.md</code>	User — Applies to all projects
<code>.github/copilot-instructions.md</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code>	Project — All sessions in this project
<code>.github/instructions/*.instructions.md</code>	Project — Glob pattern in frontmatter <code>applyTo: "**/*.ts,**/*.tsx"</code>

Claude Code

File	Scope
<code>~/.claude/CLAUDE.md</code>	User — Applies to all projects
<code>CLAUDE.md</code> (project root)	Project — All sessions in this project
<code>subdir/CLAUDE.md</code>	Project — Loaded when Claude works in that folder
<code>.claude/rules/*.md</code>	Project — Glob pattern in frontmatter <code>paths: ["src/**/*.ts,tsx"]</code>

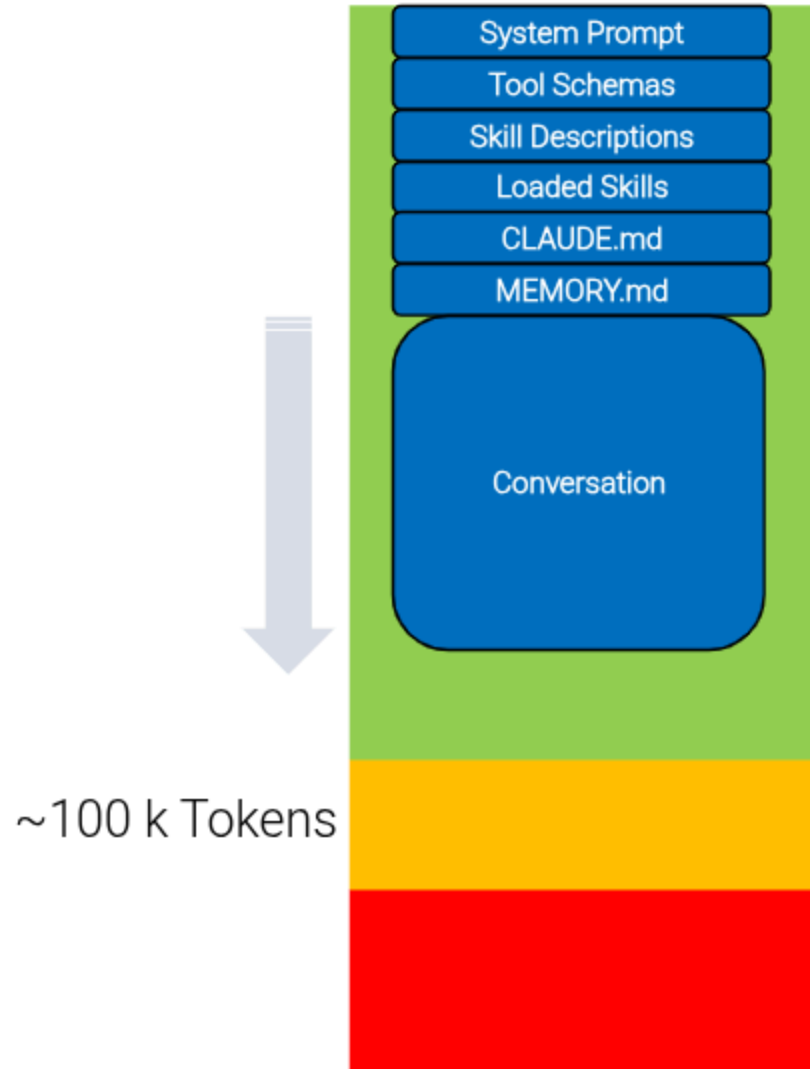
Context Management

Context is prepared and sent with *every* request.

Content	Description	Example
System Prompts	Define the system behaviour.	"You are Claude Code ..."
Tool Schemas	JSON schema of all tools: description, parameters, result	<code>Read</code> : (file_path, limit, offset)
Skill Descriptions	Name and description of all skills	<code>code-review</code> : "Does a code review. Use when ..."
Loaded Skills	Loaded skill prompts	<code>code-review</code> : "Do a strict code review."
<code>CLAUDE.md</code> and <code>.claude/rules/*.md</code>	Project specific instructions	"Use hexagonal architecture and TDD ..."
<code>MEMORY.md</code>	Persisted memory managed by Claude Code.	"npm run dev starts the server"
Conversation Turns	All messages: user, assistant, tool calls and results	User: "Summary of Main class"
		ToolCall: "READ Main.java"
		ToolResult: "public class Main { ... }"
		Assistant: "Main is printing Hello world ..."

When the context grows over ~ 100k tokens, the model performance starts to degrade.

Context Management



Context Management Commands

Long sessions degrade quality – actively manage your context.

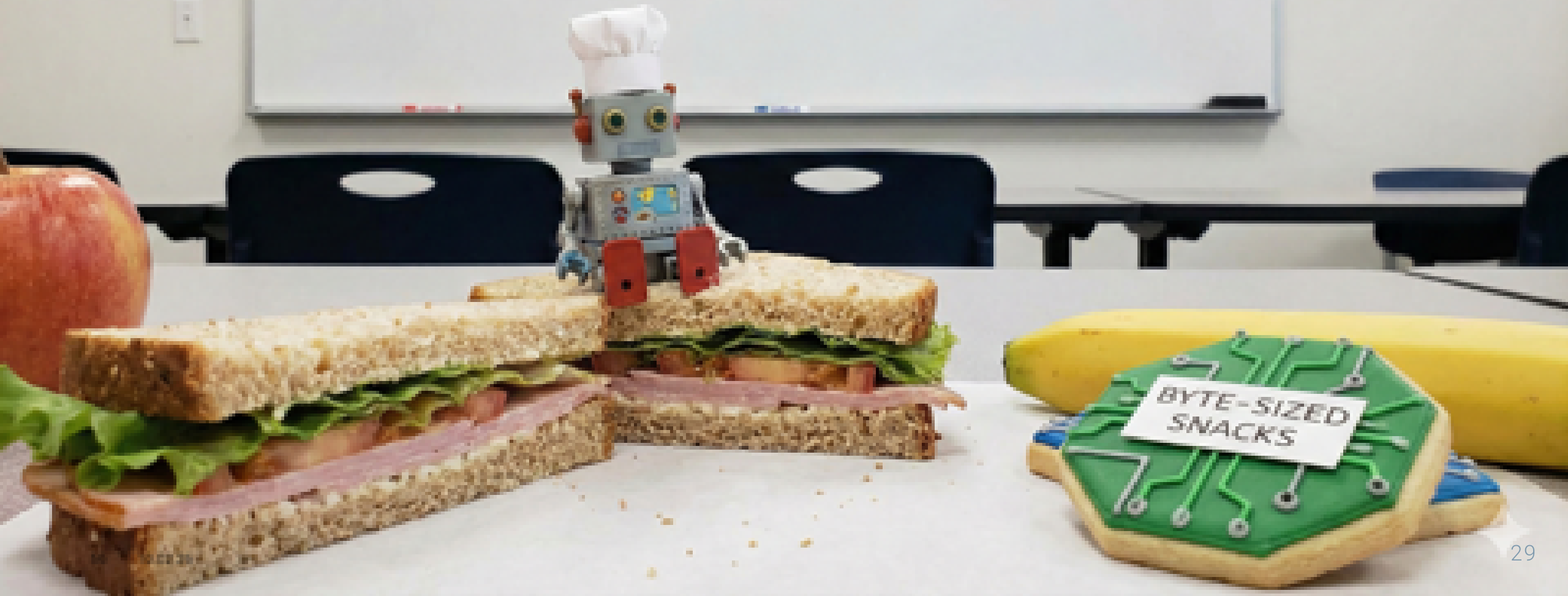
Copilot CLI and Claude Code

Command	Purpose
<code>/context</code>	Show current context window token consumption
<code>/compact <instructions></code>	Manually compact – specify what to preserve
<code>/clear</code>	Start a fresh session; use between task phases
<code>/resume</code>	Resume a previous session
<i>(auto-compaction)</i>	At ~80% capacity, Copilot automatically compacts conversation history

Key principle (both tools):

- Keep context <100k tokens.
- Keep important state in instruction files, not conversation history.
- Break tasks into smaller steps
- Reset with `/clear` when switching topics
- Use `/compact` to preserve key info if you overrun your session.

AI COURSE: LUNCH BREAK



Copilot CLI: Skills, Agents & MCP

Slash Commands

Copilot CLI	Purpose	Claude Code
<code>/model</code>	Switch model mid-session	<code>/model</code>
<code>/context</code>	Context window usage	<code>/context</code>
<code>/usage</code>	Usage activity over time	<code>/usage</code>
<code>/fleet</code>	Parallel subagents for one task	<code>/batch</code>
<code>/delegate</code>	Hand off to cloud coding agent	n/a
<code>/research</code>	Trigger deep-research agent	Explore subagent / custom skill
<code>/resume</code>	Resume previous session	<code>/resume</code>
<code>/mcp add</code>	Add an MCP server	<code>claude mcp add</code>
<code>/skills</code>	Show available skills	<code>/skills</code>
<code>/agent</code>	Manage agents	<code>/agents</code>
<code>/clear</code>	Start a new session	<code>/clear</code>
<code>/compact</code>	Compress conversation history	<code>/compact</code>
<code>/add-dir</code>	Grant directory access	<code>/add-dir</code>
<code>/help</code>	List all commands	<code>/help</code>

Both tools use `/` as the command prefix inside interactive sessions.

Skills

Keep repeating the same prompt? Skills let you name and reuse it.

Installing skills

Level	Copilot CLI	Claude Code
User	~/.copilot/skills/	~/.claude/skills/
Project	.github/skills/	.claude/skills/

Example `explain-code/SKILL.md`

```
---
name: explain-code
description: |
  Explains code with diagrams and analogies.
  Use when teaching how code works or asked "how does X work?".
---
When explaining code: start with an analogy, draw a diagram, walk through step-by-step.
```

Careful: `$ARGUMENTS` substitutions are Claude Code only.

Skills can be invoked by the user with `/explain-code` or by the model if the description matches the task.

Skills - Frontmatter Fields

Field	Copilot CLI	Claude Code
<code>name</code>	✓ required	✓ optional (default: dir name)
<code>description</code>	✓ required	✓ recommended
<code>allowed-tools</code>	✓	✓
<code>user-invocable</code>	✓ allow user-invoke	✓ allow user-invoke
<code>disable-model-invocation</code>	✓ prevent auto-invoke from model	✓ prevent auto-invoke from model
<code>model</code>	—	✓ model to use or <code>inherit</code>
<code>effort</code>	—	✓ <code>low</code> / <code>medium</code> / <code>high</code> / <code>max</code>
<code>context: fork</code> / <code>agent</code>	—	✓ run as isolated subagent
<code>argument-hint</code>	—	✓ argument description in <code>/</code> menu
<code>arguments</code>	—	✓ named <code>\$arg</code> substitution
<code>paths</code>	—	✓ path specific rules when to trigger
<code>when_to_use</code>	—	✓ extra trigger phrases

Skills - Additional Files

Place additional reference files and scripts alongside `SKILL.md` — reference them from the skill file.

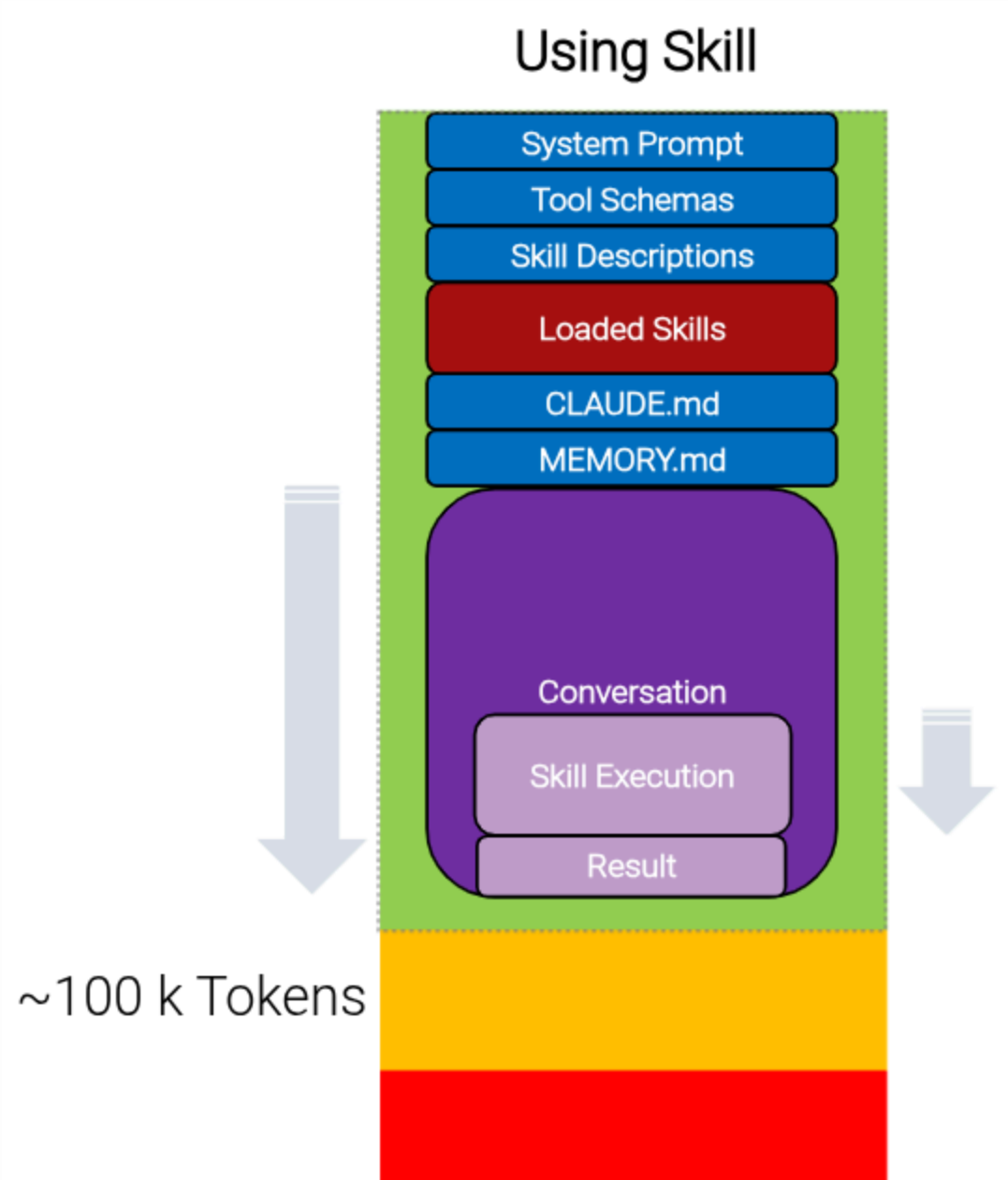
```
.claude/skills/  
└─ write-releasenote/  
    └─ SKILL.md  
    └─ docu-releasenotes.adoc  
    └─ get-changes.sh
```

Example `SKILL.md` and `get-changes.sh`

```
---  
name: write-releasenote  
description: |  
  Write a release note.  
allowed-tools: Bash  
---  
Run script `get-changes.sh` to collect the relevant git history.  
Write the release note based on the output.  
For reference: `docu-releasenotes.adoc`
```

```
#!/usr/bin/env bash  
set -euo pipefail  
  
LAST_TAG=$(git describe --tags --abbrev=0 2>/dev/null || echo "")  
if [ -n "$LAST_TAG" ]; then  
  git log "$LAST_TAG"..HEAD --oneline  
else  
  git log --oneline -20  
fi
```

Skills – Context Management



Skills - Types of Skills

Skills can be roughly categorized based on their purpose.

- **Knowledge** — give the agent domain facts or context
e.g. a style guide, or a library's known footguns
- **Workflow** — prescribe a repeatable, step-by-step procedure
e.g. a deployment or security-review routine
- **Tool** — teach the agent how to use a specific tool well
e.g. driving Playwright, or filling PDF forms

All three are still just a folder with a `SKILL.md` — the type is about what you put inside, not a different feature.

Group Exercise – Word Counting Skill

🕒 Time: 10 minutes

We build a simple word counting skill together.

Instructions

1. Create `.claude/skills/word-count/SKILL.md`
2. Add frontmatter: `name`, `description` (when to trigger)
3. Write instructions: count words in one or multiple files, report a per-file count and a total
4. Ask Claude to run the skill on a file – watch it auto-trigger from your `description`

```
---
name: word-count
description: Count words in one or more files. Use when the user asks
  for a word count or how many words a file contains.
---
```

```
Count the words in each file the user provides.
Report a per-file count and a combined total.
```

Agents & Subagents

Split complex tasks across specialized agents running in parallel in isolated contexts.

Installing agents

Level	Copilot CLI	Claude Code
User	~/.copilot/agents/	~/.claude/agents/
Project	.github/agents/	.claude/agents/

Example `~/.claude/agents/log-analyzer.md`:

```
---
name: log-analyzer
description: Analyzes long log files and verbose command output, reporting the key errors, warnings, and patterns. Use when a log or build output needs to be summarized.
model: haiku
---

You are a log analysis specialist. When given a log file or command output:
1. Scan for errors, warnings, and failures
2. Group recurring entries and identify the likely root cause
3. Return a concise summary of the key issues, where they occur, and a suggested next step.

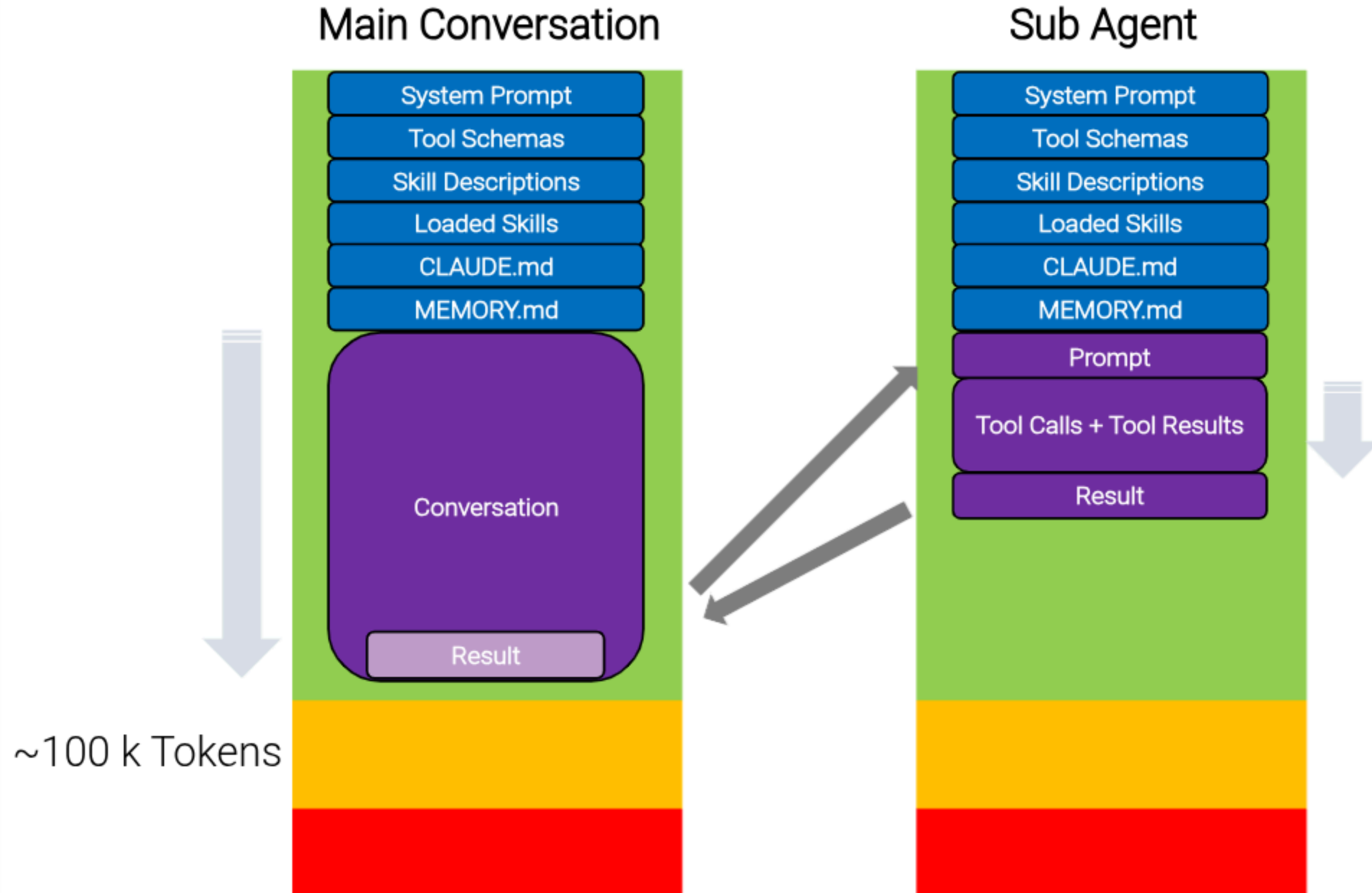
Report only your findings – do not echo the raw log.
```

Each subagent runs in a fresh, isolated context window and uses its own tokens – only its final result returns to the main agent.

Agents - Frontmatter Fields

Field	Copilot CLI	Claude Code
<code>name</code>	✓ required	✓ required
<code>description</code>	✓ required (triggers delegation)	✓ required (triggers delegation)
<code>tools</code>	✓ restrict tool access	✓ tools subagent can use
<code>disallowedTools</code>	—	✓ tools to exclude
<code>model</code>	✓ model to use	✓ <code>sonnet</code> / <code>opus</code> / <code>haiku</code> / <code>inherit</code>
<code>user-invocable</code>	✓ allow user-invoke	—
<code>disable-model-invocation</code>	✓ prevent auto-invoke from model	—
<code>skills</code>	? undocumented array	✓ load skills at startup
<code>mcpServers</code>	—	✓ available MCP servers
<code>mcp-servers</code>	✓ available MCP servers	—
<code>permissionMode</code>	—	✓ <code>default</code> / <code>acceptEdits</code> / <code>bypassPermissions</code> / <code>dontAsk</code> / <code>auto</code> / <code>plan</code>
<code>maxTurns</code>	—	✓ max agentic turns
<code>background</code>	—	✓ always run async
<code>isolation</code>	—	✓ <code>worktree</code> — isolated git context
<code>color</code>	—	✓ display color
<code>initialPrompt</code>	—	✓ initial prompt sent to agent
<code>memory</code>	—	✓ <code>user</code> / <code>project</code> / <code>local</code>

Agents – Context Management



Group Exercise – Code Reviewer Sub-agent

🕒 Time: 10 minutes

We build a focused code-reviewer sub-agent together.

Instructions

1. Create `.claude/agents/code-reviewer.md`
2. Add frontmatter: `name`, `description` (when to trigger), `tools` (read-only: `Read, Grep, Glob`), and optionally `model`
3. Write the body – this is the sub-agent's **system prompt**: its role, what to check, and how to format findings

```
---
name: code-reviewer
description: Reviews code for quality, bugs, and security issues. Use when the users asks for a code review.
---

You are a code reviewer.
Analyze code and provide actionable feedback on quality and security.

Report findings grouped by priority:
- Critical (must fix)
- Warnings (should fix)
- Suggestions (nice to have)
```

MCP – Model Context Protocol

Open standard by Anthropic for connecting AI agents to external tools and data sources.

Architecture



What MCP servers expose

Primitive	Description	Example
Tools	Functions the AI can call	<code>create_issue</code> , <code>run_query</code>
Resources	Read-only data sources	files, DB rows, API responses
Prompts	Reusable prompt templates	<code>summarize-pr</code> , <code>explain-error</code>

- **Write once, works everywhere** – one MCP server runs in Copilot, Claude Code, Cursor, any host
- **Discovery at runtime** – agent asks the server what it can do; no hardcoded tool lists
- **Ownership shifts** – integration lives with the tool owner, not the AI vendor

MCP Integration

Copilot CLI

- **GitHub MCP built-in** — No config needed; Copilot can search issues, PRs, labels, repos natively
- Add more servers with `/mcp add` inside a session

```
/mcp add context7
```

In the MCP terminal dialog add the executable command:

```
npx -y @upstash/context7-mcp
```

Claude Code

```
claude mcp add --global context7 -- npx -y @upstash/context7-mcp
```

MCP servers can be a significant security risk. Never blindly install remote MCP servers. Raiffeisen maintains a list of allowed MCP servers.

Integrating External Tools – MCP vs. CLI + Skill

Consider using a CLI + skill instead of a full MCP integration.

Advantages CLI + skill

- **Simple setup** – CLI usually simple to install; skill is one markdown file
- **LLMs know common CLIs** – trained on `gh`, `kubectl`, `aws` docs; no schema needed
- **Specific CLIs** – Some providers publish CLI and skill specific for agentic integration (e.g. `playwright-cli`)

Pattern: CLI + optional skill

Step 1 – Use the CLI directly – `gh`, `kubectl`, `aws`, ... work via `Bash` immediately.

Step 2 – Write a skill (optional) – teaches the agent project-specific conventions.

```
---
name: github-workflow
description: Create PRs, review issues, and manage GitHub tasks for this project
tools: Bash
---
Use the `gh` CLI for all GitHub operations.
Create PR: `gh pr create --fill --assignee @me`
List issues: `gh issue list --label bug --state open`
Check CI:  `gh run list --branch $(git branch --show-current) --limit 5`
Our repo uses conventional commits. Always link issues in PR body with "Closes #N".
```

Spec Driven Development

Agentic Coding for Software Engineers

Vibe Coding – And Why It Breaks Down

Vibe Coding works for prototypes and throwaway code, but breaks down at scale.

Where It Breaks

- Multi-session tasks: the model loses track of decisions made two sessions ago
- Team tasks: no shared artifact representing what was agreed
- Complex features: requirements drift as the conversation grows
- Design misalignment: model infers architecture and patterns from context – silently diverging from developer intent

Vibe coding optimizes for fast initial results, spec-driven development optimizes for the long-term maintainability.

Spec Driven Development (SDD)

Spec Driven Development is a methodology where formal specifications serve as the source of truth for AI code generation.

Shift from "Vibe Coding"

- Moves away from unstructured, chat-based ad-hoc prompting where requirements are lost in conversation history
- Prioritizes "**Documentation First**" — specs are written and approved before a single line of code is generated

Core Philosophy

- **Intent over Implementation** — Humans define what and why (Intent)
- **AI as Implementer** — AI Agents handle the how (Implementation) based on strict boundaries defined in the spec
- **Living Artifacts** — Specifications are not static; they evolve with the software and remain the source of truth for future maintenance

SDD as Context Engineering

SDD is a formalized, persistent form of context engineering.

- **Structured Control** — SDD locks context down into a validated specification document before coding begins
- **Ephemeral Context, Persistent Spec** — Sessions come and go; the spec survives all of them
- **Prevents Drift** — New sessions start from the spec, not from vague memory of previous chat
- **Context Hygiene** — Starting SDD workflow steps with a fresh context prevents derailing agents due to context rot

The SDD Loop

SDD provides a workflow that guides you through structured specification-driven development.

1. **Specify** (*The "What"*) – Human provides high-level intent. AI drafts a detailed specification (requirements, acceptance criteria).
2. **Review** (*The Gate*) – Human validates the Spec.
3. **Plan** (*The "How"*) – AI converts the Spec into a technical implementation plan (files to touch, APIs to create, tasks checklist).
4. **Implement** (*The Action*) – AI executes the plan, creating code and tests. Marks tasks **[x]** as it goes.
5. **Verify** – Automated tests and human review ensure code matches the original Spec, not just "works".

The requirement and task specs prevent AI derailment during implementation.

Minimal SDD Framework

A simple framework to apply SDD.

1. `/clear` + `/sdd-clarify` + `/sdd-write-requirements` — Align with the AI and write requirements spec
2. `/clear` + `/sdd-write-tasks` — Break the spec into a task list with acceptance criteria
3. `/clear` + `/sdd-implement-task` — For each task, start a fresh session and implement it with the spec as context
4. Repeat 3. until all tasks are done

The `/sdd-clarify` is a powerful tool to align your understanding with the AI before writing the spec. This ensures that the generated spec and implementation tasks do not need to make assumptions about design decisions.

Do not use sdd-* skills in Plan Mode — Plan Mode reframes the task as plan-then-implement and can skip the SDD steps that write the requirements and task specs.

Exercise – Spec Driven Development

Project: Spring boot, backend, angular frontend, minimal features, running Copilot and Claude ready: CLAUDE.md, skills, ...

Implement a feature end-to-end (backend and frontend).

Use Copilot CLI (`copilot`) – or Claude Code (`claude`) if you prefer.

 Time: 90 minutes

Setup

- Clone the project according to the instructions.
- Read through the `README.md` and familiarize yourself with the project structure.
- `mvn clean package && mvn -pl order-service spring-boot:run` to start. Browse `http://localhost:8080`.
- Ask the agent some questions about the project to get a feel for how it responds and what context it has.
- Follow the instructions in the following file:
 - `exercise/README-skill-based-spec-driven-development.md`

Some things to look out for

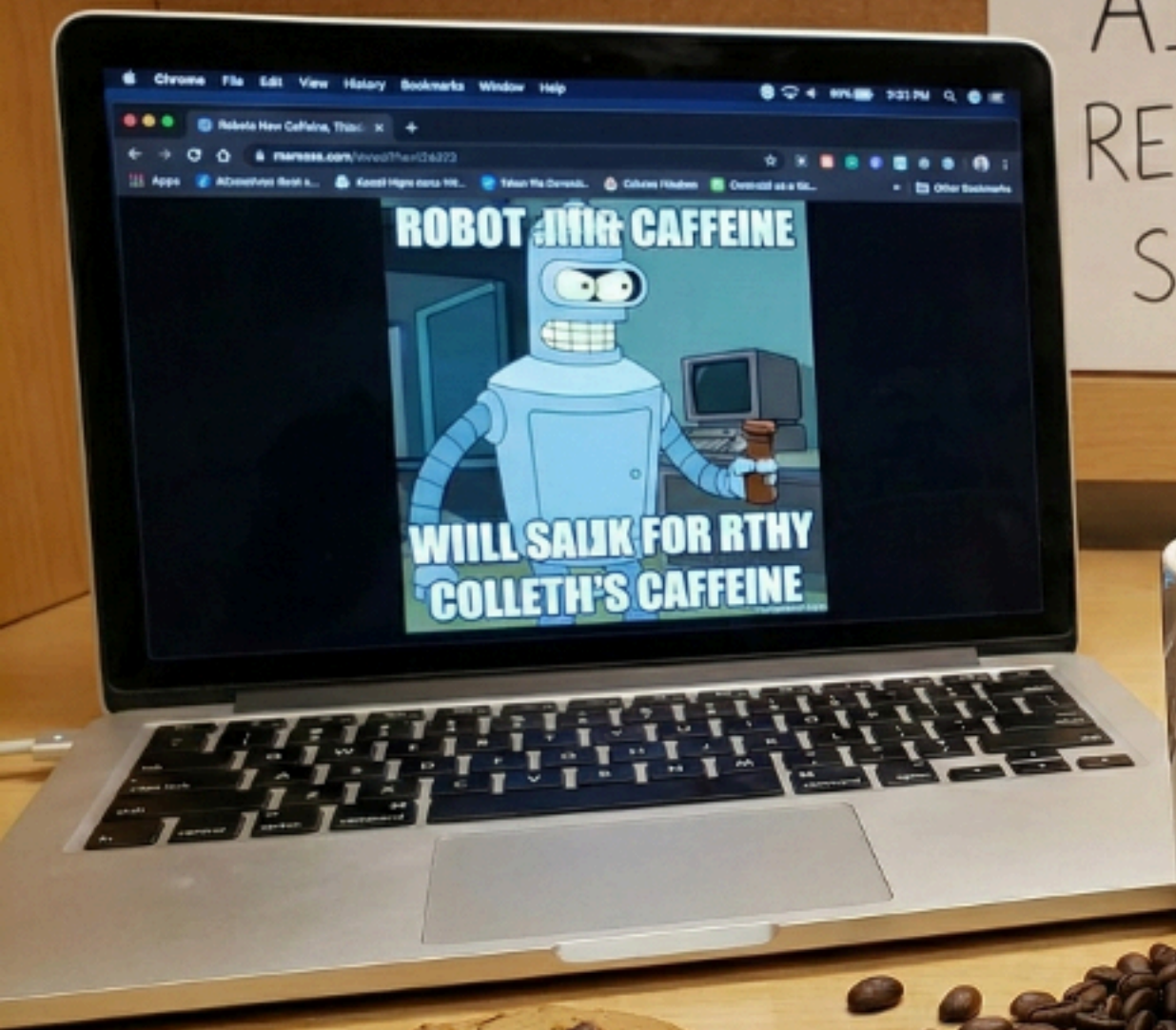
- Make sure you are NOT in auto-pilot mode of `copilot`
- Review the generated `prd.md`
 - Is the frontend defined?

Reflection (10 min)

Discuss with the group:

- How does this differ from vibe coding the same ticket?
- What are the advantages of persisting task state in a file vs. relying on conversation context?
- Why did we start a fresh session before every task implementation phase?

AI LAB: REFUELING STATION



Wrap-up

Agentic Coding for Software Engineers

Key Takeaways

Five things to carry with you from today.

1. **Skills and agents multiply leverage** — One skill file eliminates repeated prompting; subagents split complex tasks across isolated contexts in parallel
2. **Context failures cause most AI failures** — Long loops fill the window, compaction loses detail; project instructions and SDD keep context clean across sessions
3. **Verify at the boundaries** — Review diffs for deleted tests, changed signatures, and scope creep; you own every committed line
4. **Project instructions are your foundation** — Set up `CLAUDE.md` / `copilot-instructions.md` before any substantial work; it's the difference between a confused agent and a focused one.*
5. **SDD for non-trivial tasks** — For anything that spans multiple sessions or files, write a spec first; it prevents drift and enables safe session resets between phases

The goal is not to write less code — it's to ship better software faster while staying in control.

What Next

Experiments for the coming weeks.

This Week

- Set up a `CLAUDE.md` for one of your current projects – works in both Copilot CLI and Claude Code

This Month

- Apply the SDD loop to one real ticket: spec → plan → implement
- Explore one MCP that connects to a tool you use daily (JIRA, Confluence, GitHub)
- Build one custom skill for a workflow you repeat often

When you migrate to Claude Code

- Re-add MCP servers with `claude mcp add`
- Skills in `.claude/skills/` work in both tools – no migration needed

Ongoing

- Stay skeptical – verify AI output with the same rigor as any team member's PR
- Share what works (and what doesn't) with your team

Useful Links

- **Claude Code Docs** — <https://code.claude.com/docs/en/overview>
- **Claude API Docs** — <https://docs.anthropic.com>
- **Anthropic Courses** — <https://anthropic.skilljar.com/>
- **MCP Protocol** — <https://modelcontextprotocol.io>
- **Claude Cookbooks** — <https://github.com/anthropics/claude-cookbooks>
- **GitHub Copilot Docs** — <https://docs.github.com/en/copilot>

Q&A / Open Discussion

Topics we can go deeper on:

- Any section from today you want to revisit
- Questions about applying these tools to your specific codebase
- Concerns or skepticism worth discussing openly

Feedback:

- What was most valuable today?
- What was missing or unclear?

Thank You!